

Lab 4 — XSS, Session Management and Authentication Resilience Assessment

Student: Jamiu Afolabi

Reg No: C11/26/EHIT/17319

Course: Offensive Security I

Assessment: Lab 4 of 5

Assessment Date: 3/09/2026

Environment: Kali Linux / DVWA / OWASP Mutillidae II

1. Introduction

1.1 Assessment Overview

This assessment was carried out in a control environment to test out the XSS, Session management and authentication resilience for two web applications - DVWA and Mutillidae II. The aim is to understand how XSS work, validate them using control testing, and compare their response at different security levels. It also tested session security and cookie rotation.

1.2 Scope and Rules of Engagement

Testing was restricted to the assigned local/host-only DVWA and OWASP Mutillidae II applications.

Only learner-controlled accounts and harmless XSS markers were used. No cookie theft, session hijacking, phishing overlays, keystroke capture, token collection, credential stuffing, password spraying or testing against real accounts was performed.

Authentication testing was limited to a maximum of five controlled failed attempts per target.

1.3 Laboratory Environment

Component	Details
Operating System	Kali Linux
Target 1	DVWA
Target 2	OWASP Mutillidae II
Deployment	Docker
Network	Local / Host-only laboratory environment
Testing Tools	Kali

DVWA URL	http://127.0.0.1:4280/login.php
Mutillidae II URL	https://127.0.0.1

1.4 Testing Methodology

Testing followed a controlled process. Normal application behaviour was first established before introducing a minimal test input.

For XSS, harmless visual markers were used to determine whether user-controlled input was reflected, stored or processed through a client-side DOM sink.

For session management, cookie and session behaviour was examined before login, after login and after logout.

Authentication resilience was assessed using one dedicated learner-controlled account and no more than five controlled failed attempts per target.

2. XSS Assessment

2.1 XSS Testing Overview

The XSS assessment examined reflected, stored and DOM-based injection behaviour in DVWA and Mutillidae II.

Testing focused on identifying the input source, output sink, rendering context and resulting browser behaviour.

2.2 DVWA Reflected XSS

Baseline

Security Level: Low

Request/URL: http://127.0.0.1:4280/vulnerabilities/xss_r/?name=It%27s+Lab+4+time#

Affected Parameter: name

Normal Input: It's Lab 4 time

Observed Behaviour: The submitted value was reflected in the page response as:

Hello It's Lab 4 time

Result: The input was reflected by the application.

Evidence: E-01-DVWA-Reflected-XSS-baseline

Harmless Execution Proof

Test Input/Marker: `<script>alert('It's Lab 4 XSS Session')</script>`

Observed Behaviour: A browser alert appeared displaying **It's Lab 4 XSS Session.**

Rendering Context: The supplied input was reflected into the HTML response and interpreted as executable JavaScript by the browser.

Result: Vulnerable

Evidence: E-02-DVWA-Reflected-XSS-Proof

2.3 DVWA Stored XSS

Baseline

Security Level: Low

Request/URL: http://127.0.0.1:4280/vulnerabilities/xss_s/

Affected Parameter: Message

Name: Jamiu

Message: This is lab 4 Stored XSS baseline test

Normal Input:

Observed Behaviour: The comment was submitted successfully and displayed on the comment page.

Result: Normal comment stored and rendered successfully.

Evidence: E-03-DVWA-Stored-XSS-baseline

Harmless Execution Proof

Test Input/Marker: `<script>alert("This is lab 4 stored XSS")</script>`

Observed Behaviour: The application display a browser alert with the text "This is lab 4 stored XSS"

Persistent: Confirmed. Since it's stored XSS, if the URL is copied to another tab or window, the application still shows the browser alert that displays "This is lab 4 stored XSS".

Result: Vulnerable

Evidence: E-05-DVWA-Stored-XSS-Proof

Second-session Persistence

Verification Method: Second user-controlled browser session (private window)

Stored Marker: `<script>alert("This is lab 4 stored XSS")</script>`

Observed behaviour: The stored marker executed again when the Stored XSS page was opened in the private browser session.

Persistence: Confirmed

Result: Stored XSS confirmed

Evidence: E-05-DVWA-Stored-XSS-Persistence-Proof

2.4 DVWA DOM XSS

Baseline

Security Level: Low

Normal Behaviour: The page displays a **Select a language** drop-down containing the available language options.

Source: `document.location.href`

Input: `default=` URL parameter

Processing: The value following `default=` is extracted and passed through `decodeURI()`.

Sink: `document.write()`

Rendering Context: HTML `<option>` element.

Result: A client-side data flow from the URL to an HTML-writing sink was identified.

Evidence: E-06-DVWA-DOM-XSS-source-sink, E-07-DVWA-DOM-XSS-Baseline

Harmless Execution Proof

Test Input/Marker: `<script>alert('This is lab 4 DOM XSS session')</script>`

Observed Behaviour: The application accepted the input via the “default=” parameter and returned a browser alert showing the text passed into it - This is lab 4 DOM XSS session.

Result: Vulnerable - DOM XSS confirmed

Evidence: E-08-DVWA-DOM-XSS-proof

2.5 Mutillidae II XSS Testing

Baseline

Security Level: 0 (Hosed)

Function: Echo Message

Normal Input: hello

Observed Behaviour: The application reflected **hello** back onto the page.

Result: Normal input reflected successfully.

Evidence: E-12-Mutillidae-Reflected-XSS-baseline

Reflected XSS

Request/URL: https://127.0.0.1/index.php?page=echo.php

Affected Parameter: Echo Message field

Input/Marker: <script>alert('LAB4-Mutillidae-XSS')</script>

Rendering Context: LAB4-Mutillidae-XSS'

Observed Behaviour: It responded with a browser alert displaying the text "LAB4-Mutillidae-XSS"

Result: Vulnerable - Reflected XSS Confirmed

Evidence: E-13-Mutillidae-Reflected-XSS-proof

Stored XSS

Baseline

Security Level: 0 (Hosed)

Function: Add Blog

Normal Input: Its Lab 4 Mutillidae II Stored/Persistent XSS test

Observed Behaviour: The blog entry was submitted and displayed by the application.

Result: Normal blog content stored and rendered successfully.

Evidence: E-14-Mutillidae-Stored-XSS-baseline

Harmless Marker Execution

Input Location: Add Blog

Input/Marker: <ScRiPt>alert("This is Mutillidae Lab4 Stored XSS test")</ScRiPt>

Persistence Observed: Yes

Second-Session Verification: The second session verification confirmed the persistent nature of the Stored XSS vulnerability. I opened a private browser and navigate to the "Add Blog" section of the Stored/Persistent XSS, and the script still execute by displaying an alert with the test "This is Mutillidae Lab4 Stored XSS test"

Observed Behaviour: After inputting the script, the web application responded with a browser alert displaying the test "This is Mutillidae Lab4 Stored XSS test"

Result: Vulnerable - Stored XSS confirmed

Evidence: E-15-Mutillidae-Stored-XSS-Proof, E-16-Mutillidae-Stored-XSS-Second-Session

DOM XSS

Baseline

Security Level: 0 (Hosed)

Function: DOM-Based – HTML5 Web Storage

Normal Input: Existing Session Storage and Local Storage values

Observed Behaviour: The stored key/value pairs were displayed in the Web Storage table.

Result: Normal Web Storage content was retrieved and rendered successfully.

Evidence: E-17-Mutillidae-DOM-WebStorage-baseline

Harmless Marker Execution

Source: DOMStorageKey (Key input)

Sink: innerHTML in setMessage()

Rendering Context: LAB4-Mutillidae-DOM-XSS

Observed Behaviour: The harmless JavaScript event-handler marker; , executed and displayed the text "LAB4-Mutillidae-DOM-XSS" in a browser alert.

Result: Vulnerable - DOM XSS confirmed

Evidence: E-18-Mutillidae-WebStorage-DOM-XSS-Proof

2.6 XSS Security Comparison

Application	Weak Configuration	Strong Configuration	Result	Security Control Observed
DVWA	Low	Impossible	At low security, reflected, stored and dom executed the script and returned a browser alert. Under impossible conditions, the markers were rendered as text and didn't execute.	This used stronger input/output handling including HTML encoding prevented the supplied script marker from being interpreted and executed as JavaScript.
Mutillidae II	0 (Hosted)	5 (Secure)	At Level 0, reflected, stored and DOM XSS were confirmed. At Level 5, reflected and stored XSS markers did not execute and were blocked or altered by the application's filtering/encoding behaviour. However, DOM XSS remained vulnerable through the innerHTML sink after the input limit was increased for controlled testing.	Level 5 applied filtering/encoding that prevented the tested reflected and stored markers from executing. However, the vulnerable DOM innerHTML output path remained available, so the stronger configuration did not fully prevent XSS.

3. Session Management Assessment

3.1 Session Lifecycle

DVWA

Pre-Login Session: A PHPSESSID session cookie was present before authentication.

Post-Login Session: An authenticated session was established, allowing access to protected DVWA functionality. The session cookie had HttpOnly=False, Secure=False, SameSite=None, Path=/, and a defined expiry.

Session Identifier Rotated: Not explicitly verified during testing.

Post-Logout Session: After logout, attempting to access an authenticated page returned the user to the login page.

Old Session Still Valid: No. The authenticated session was no longer accessible through normal browser navigation after logout.

Mutillidae II

Pre-Login Session: A session cookie was present before authentication.

Post-Login Session: An authenticated session was established. The session cookie had HttpOnly=False, Secure=False, SameSite=None, Path=/, and a defined expiry.

Session Identifier Rotated: Not explicitly verified during testing.

Post-Logout Session: After logout, attempting to access an authenticated page required the user to log in again.

Old Session Still Valid: No. The authenticated session was no longer accessible through normal browser navigation after logout.

3.2 Cookie Security

DVWA

Cookie	Secure	HttpOnly	SameSite	Domain	Path	Expiry
PHPSESSID	False	False	None	Not recorded	/	04 September 2026 14:27:34 GMT

Evidence: E-19-DVWA-Cookie-Security

Mutillidae II

Cookie	Secure	HttpOnly	SameSite	Domain	Path	Expiry
PHPSESSID	False	False	None	Not recorded	/	04 September 2026 14:27:34 GMT

Evidence: E-20-Mutillidae-Cookie-Security

3.3 Logout Invalidation

DVWA

Logout Performed: Yes

Previous Session Reused: Yes

Observed Result: After logout, an attempt to access an authenticated page returned the user to the login page, indicating that the authenticated session was no longer valid.

Mutillidae II

Logout Performed: Yes

Previous Session Reused: No

Observed Result: After logging out and attempting to access an authenticated page, the app redirects to the login page, meaning the initial authenticated session was no longer valid.

3.4 Session Security Comparison

Control	DVWA Low	DVWA Impossible	Mutillidae II
Session rotation at login	No	Yes	No at all level
Session invalidation at logout	Yes	Yes	Yes
Secure cookie	False	False	False
HttpOnly	False	True	False
SameSite	None	Strict	None
Appropriate path/domain	Path /; Domain not recorded		Path /; Domain not recorded

DVWA: At **Impossible**, the PHP session identifier changed following authentication, demonstrating session rotation.

Mutillidae II: At **Security Level 5 (Secure)**, the PHP session identifier remained unchanged before and after authentication, so session rotation was not observed.

4. Authentication Resilience

4.1 Controlled Authentication Test

Testing was restricted to one dedicated learner-controlled account per target and a maximum of five failed attempts.

DVWA

Test Account: username: admin, password: admin

Attempt	Time	Result	Delay / Lockout / Alert
1	Not recorded	Login failed	Immediate response; no delay, lockout or additional alert observed
2	Not recorded	Login failed	Immediate response; no delay, lockout or additional alert observed
3	Not recorded	Login failed	Immediate response; no delay, lockout or additional alert observed
4	Not recorded	Login failed	Immediate response; no delay, lockout or additional alert observed
5	Not recorded	Login failed	Immediate response; no delay, lockout or additional alert observed

Rate Limiting Observed: No

Account Lockout Observed: No

Alert/Warning Observed: No]

Conclusion: Five controlled failed authentication attempts received immediate responses at both Low and Impossible security levels. No observable rate limiting, account lockout, or additional authentication warning was triggered.

Mutillidae II

Test Account: username: jay password: test

Attempt	Time	Result	Delay / Lockout / Alert
1	Not recorded	Login failed — “Username or password incorrect”	Immediate response; no delay, lockout or additional alert observed
2	Not recorded	Login failed — “Username or password incorrect”	Immediate response; no delay, lockout or additional alert observed
3	Not recorded	Login failed — “Username or password incorrect	Immediate response; no delay, lockout or additional alert observed
4	Not recorded	Login failed — “Username or password incorrect”	Immediate response; no delay, lockout or additional alert observed
5	Not recorded	Login failed — “Username or password incorrect”	Immediate response; no delay, lockout or additional alert observed

Rate Limiting Observed: No

Account Lockout Observed: No

Alert/Warning Observed: No

Conclusion: Here, five controlled failed authentication sessions were attempted and they all receive immediate response at secure level 0 and 5. No observable rate limiting, account lockout, or additional authentication warning was triggered.

5. Security Analysis

5.1 XSS Analysis

What distinguishes reflected, stored and DOM XSS?

Reflected XSS runs from the current request, stored XSS is saved by the application and shown to users, while DOM XSS happens through changes made to the page by JavaScript.

Why is a harmless visual marker sufficient to prove browser execution?

A harmless visual marker shows that the browser executed the injected script without causing damage.

5.2 Session Management Analysis

Why should a session rotate at login and become invalid after logout?

The session should rotate at login to prevent session fixation and become invalid after logout to prevent session reuse.

5.3 Authentication Analysis

How did the applications respond to repeated failed authentication attempts?

The applications rejected the incorrect login attempts. We stopped after the limited number of attempts required for the test.

What protections were observed?

Login validation was observed, and the testing was limited to a small number of failed attempts on a learner-controlled account.

How could account lockout itself create an availability risk?

An attacker could intentionally cause repeated failed logins and lock a user out of their account.

5.4 Cross-Target Comparison

Security Area	DVWA	Mutillidae II	Better Protection / Observation
Reflected XSS	Vulnerable at Low; marker did not execute at Impossible	Vulnerable at Level 0; marker was rejected at Level 5	DVWA and Mutillidae both showed effective protection by rejecting the marker in strong configuration.
Stored XSS	Vulnerable at Low; marker did not execute at Impossible	Vulnerable at Level 0; marker was escaped/altere d and did not execute at Level 5	Both provided protection in the strong configuration.
DOM XSS	Vulnerable at Low; marker did not execute at Impossible	Vulnerable at Level 0 and remained vulnerable at Level 5	DVWA provided better protection
Output encoding	Stronger encoding/handling observed at Impossible	Filtering/escapi ng observed for reflected and stored XSS, but vulnerable innerHTML path remained for DOM XSS	DVWA showed more protection across the tested XSS cases.

Session rotation	Rotated at Impossible	Did not rotate at Level 5	DVWA provided better protection.
Logout invalidation	Yes	Yes	Both invalidated the authenticated session after logout.
Cookie security	Weak configuration at Low: HttpOnly=False, SameSite=None, Secure=False. At Impossible: HttpOnly=True, SameSite=Strict, Secure=False	At all security level: Secure=False, HttpOnly=False, SameSite=None	DVWA provided better cookie protection in the strong configuration, with HttpOnly=True and SameSite=Strict, while Mutillidae retained weaker observed cookie attributes at Level 5.
Rate limiting	Not observed at Low or Impossible	Not observed	Neither application demonstrated rate limiting during the controlled five-attempt test.

6. Remediation

Finding	Recommended Remediation
Reflected XSS	Apply context-aware output encoding before rendering user-controlled data.
Stored XSS	Validate input where appropriate and encode output according to the rendering context.
DOM XSS	Avoid unsafe DOM APIs and use safe APIs such as <code>textContent</code> where applicable.
Session fixation	Rotate the session identifier after successful authentication.
Session reuse after logout	Invalidate the authenticated server-side session on logout.
Insecure cookies	Use appropriate <code>Secure</code> , <code>HttpOnly</code> and <code>SameSite</code> attributes.
Weak authentication resilience	Implement rate limiting, progressive delays and appropriate monitoring.
Password attacks	Use MFA and strong password controls.
Excessive lockout	Balance lockout controls with rate limiting and monitoring to reduce denial-of-service risk.

7. Evidence-Minimisation and Cleanup

Only the minimum evidence required to demonstrate each finding was collected.

No cookies, session tokens, credentials or other sensitive authentication material were unnecessarily retained or shared outside the protected evidence package.

Cleanup

- Stored XSS markers removed: Yes
- Learner sessions logged out: Yes
- Dedicated browser profiles cleared: Yes
- Test accounts reset/unlocked: Yes
- Temporary files removed: Yes
- Tokens/cookies removed from evidence where unnecessary: Yes
- Final environment restored: Yes

8. Conclusion

The Lab 4 assessment tested XSS, session management and authentication security in the authorised DVWA and OWASP Mutillidae II lab environment.

The testing covered reflected, stored and DOM-based XSS, as well as session and cookie behaviour before login, after login and after logout. Authentication was also tested using a learner-controlled account with a maximum failed attempts of five sessions.

All testing stayed within the authorised lab scope. Harmless markers were used, and no real accounts, password spraying, credential theft or cookie theft were performed.

After testing, the environment was cleaned up, markers were removed, sessions were logged out and the lab was restored.

Appendix A — Activity Log

See `activity-log.md`.

Appendix B — Evidence Register

See `evidence-register.md`.

Appendix C — Supporting Evidence

See Screenshot Folder